Predicting Text from Intracranial Neural Signals

Aryan Dalal
Los Angeles, USA
Department of Mathematics
Department of Electrical & Computer Engineering
aryandalal@ucla.edu

Nikhil Dewitt
Los Angeles, USA
Department of Computer Science
nikhildewitt@ucla.edu

Aadrij Upadya
Los Angeles, USA
Department of Computer Science
aadriju01@ucla.edu

Abstract

Modern brain-computer interfaces (BCIs) for speech restoration decode neural signals directly from neural activity. Despite the advancements in machine learning techniques, speech BCIs have not yet reached the level of performance desirable for fluent communication for people with ALS or brainstem strokes. Our paper presents a variety of techniques to improve neural sequence decoding for speech BCI applications, achieving a 18.9% Phoneme Error Rate (PER). We enhance the bidirectional Gated Recurrent Unit (GRU) based architecture provided in Brain-to-Text Benchmark '24 with several key modifications. We first contribute architectural improvements before the GRU layer with LayerNorm normalization and additional post-GRU processing layers with normalization and dropout. We then optimize the training procedure with finetuned hyperparameters with learning rate warmup, learning decay schedules and AdamW optimizer. We demonstrate a speech BCI that reduces PER by 16.34% compared to the baseline GRU model that achieves a PER of $\approx 22.68\%$, suggesting that these changes demonstrate significant improvement in neural sequence decoding accuracy. We achieve these enhancements with similar training time as the baseline model thereby not compromising on computational expense. As advancements in deep learning continue, machine learning based speech BCIs designed to restore fluent communication may achieve higher performance.

1 Introduction

We study different model improvements for Speech BCI decoding and aim to decrease the Phoneme Error Rate (PER) on the validation set to evaluate neural sequence decoding performance. Specifically, we study architectural enhancements and hyperparameter fine-tuning without compromising training time.

Speech BCI decoders provide a critical avenue for communication restoration for people with Amyotrophic Lateral Sclerosis (ALS) or Brain Stem Stroke. In one study, researchers were able to demonstrate real-time decoding of sentences for a man with anarthia [7]. Recent improvements in intracortical brain interfaces have led to significant developments in speech decoding from the premotor cortex. However, recent benchmarks have focused primarily on bidirectional architectures. While bidirectional architectures produce strong performance, their applicability in clinical contexts is vastly limited due to a lack of real-time translation–a critical goal of speech BCIs.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

In an effort to work towards a BCI relevant for clinical translation, we make architectural enhancements in an unidirectional model. We do so given the need for decoding algorithms for speech neuroprostheses that are able to operate in real-time as well as with low latency. In particular, we intend to achieve a low Phoneme Error Rate without adding model complexity so that future speech neuroprostheses can be performed on-device with privacy at the center of the BCI. There are significant challenges in training a unidirectional model–Phonemes are heavily dependent on their placement next to other phonemes, and losing future context limits a model's ability to make connections in this case. Consequently, unidirectional models face noticeable performance degradation in comparison to bidirectional models. [4] However, there's also a noticeable decrease in training time.

2 Methods

2.1 Neural Dataset

The Brain-to-Text Benchmark '24 dataset [2] consists the neural activity recorded from a participant with amyotrophic lateral sclerosis (ALS) who was implanted with microelectrode arrays (MEAs) for speech neuroprosthesis research. This dataset was released as part of a competition to improve neural speech decoding performance and is described in detail in Willett et al.(2023). The neural recordings are recorded from the premotor cortex each containing spike band power and threshold crossings. These features are concatenated, resulting in 256 total features. Neural activity is provided in 20 ms time bins. To account for session-to-sesion variability, the neural features are normalized within each block, where each block contains 20-50 sentences recorded consecutively. The dataset is divided into splits as such: training set (8,800 sentences), validation set (880 sentences), and the test set (1200 sentences).

2.2 Phoneme Error Rate

The *Phoneme Error Rate* (PER) is a model evaluation metric used for this paper. We use this metric to see if the model can accurately transcribe verbatim, We compute the PER as

$$\text{PER} = \frac{S + D + I}{N}$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions and N is the total number of characters. Similar to the Phoneme Error Rate is the *Word Error Rate* (WER) which can be computed by converting the decoded phonemes into words by using a weighted finite state transducer (WFST)-based decoder. However, due to the computational costs of using a WFST decoder, we elected to simply the PER metric. It's important to note that during experiment, the PER is in fact reported as the CER.

2.3 Connectionist Temporal Classification

The Connectionist Temporal Classification loss is a type of loss function that is well suited to address classification under settings with unsegmented sequential data. In particular, in this paper, we explore labeled sequences of phonemes with no ground-truth timing and so, the CTC loss was a relevant choice. Simply, consider a mapping from X to Y where $X = [x_1, x_2, \ldots, x_T]$ describes input sequences of phonemes and $Y = [y_1, y_2, \ldots, y_U]$ describes transcripts. The CTC Loss is given by

$$\ell_{\text{CTC}}\left(Y \mid X\right) = -\ln\left[\sum_{A \in \mathcal{A}_{X,Y}} \prod_{t=1}^{T} y_t\left(p_t\left(a_t \mid X\right)\right)\right]$$

where $\mathcal{A}_{X,Y}$ is the set of all valid alignment paths that map to the target sequence Y, T is the length of the sequence X and $p_t\left(a_t \mid X\right)$ is the probability of predicting label a_t given an input sequence X at time t.

2.4 Baseline Gated Recurrent Unit (GRU)-based Model

Our baseline model follows the architecture provided in the Brain-To-Text Benchmark '24. While the original model consists of a 5-layer bidirectional GRU with 1024 hidden units per layer, we

switched it from bidirectional to unidirectional since the goal is a model that can predict phonemes without peeking ahead. The input neural activity features (256-dimensional) are first passed through Gaussian smoothing with width hyperparameter 2.0, followed by day-specific affine transformations with softsign nonlinearity to account for the session-to-session variability that occurs across data recording days. The day transformed features are then processed to create overlapping temporal windows. The windowed features are processed by a 5-layer unidirectional GRU with 1024 hidden units per direction. A dropout of 0.4 is applied between GRU layers to avoid overfitting. The final GRU hidden state is directly passed to a linear output layer producing over 40 phoneme classes, in addition to a CTC blank token.

Training for the baseline model is performed using the Adam optimizer with a learning rate of 0.02, $\epsilon = 0.1$, and an L2 weight decay of 10^{-5}. Although a linear learning rate scheduler is used, both the initial and final learning rates are set to 0.02, resulting in an effectively constant learning rate throughout training. The model is trained for $10{,}000$ batches with a batch size of 64 on an NVIDIA Tesla T4 GPU.

To improve robustness, we apply data augmentation consisting of additive white noise with standard deviation 0.8 and constant offset perturbations with standard deviation 0.2. Training minimizes the CTC loss, which enables alignment between the neural time series and corresponding phoneme sequences without requiring explicit frame-level labels.

Under these conditions, the baseline model achieves a validation phoneme error rate (PER) of approximately 22.68%.

2.5 Experimental Modifications

2.5.1 Hyperparameter Tuning

We began by systematically exploring hyperparameter modifications. Initial experiments (trials 1-4) focused on adjusting batch size, learning rate schedules, and augmentation strengths. We found that reducing batch size from 64 to 16 improved convergence, likely due to more frequent gradient updates and better gradient signal quality. It also reduced training speed per batch, likely because of the memory pressure that a larger batch size puts on the GPU. We also experimented with different learning rate decay schedules, finding that a linear decay from 0.02 to 0.008 over the course of training improved final performance compared to constant learning rates. We noticed that a sharper linear decay (i.e. ending LR of 0.002) led to premature convergence and underfitting for the same number of batches.

2.5.2 Architectural Improvements

Inspired by the Linderman Lab entry in the *Brain-to-Text* Challenge [**?**], we experimented with adding post-GRU processing layers. Our sixth trial, which incorporated these post-GRU layers and extended training, demonstrated that adding a linear layer followed by LayerNorm and dropout after the GRU stack improved overall model stability and reduced overfitting. Additionally, we applied LayerNorm to the GRU inputs, which further stabilized training dynamics and improved convergence, following strategies similar to those used in Transformer-based neural decoding architectures [**?**].

These architectural enhancements, combined with longer training, yielded substantial improvements in validation PER compared to earlier trials.

2.5.3 Optimization Modifications

We replaced the Adam optimizer with AdamW, as proposed by Loshchilov and Hutter [**?**]. This change improved generalization and reduced overfitting, most likely due to the more effective weight regularization achieved through decoupled weight decay. To compensate for the slightly stronger regularization and slower effective learning dynamics introduced by AdamW, we adopted a milder linear learning rate schedule (from 0.02 to 0.008), which preserved optimization stability while still promoting convergence.

We additionally experimented with gradient clipping (values between 1.0 and 5.0) and label smoothing ($\varepsilon = 0.05$–0.1). However, these modifications did not provide further improvements beyond the use of AdamW; in fact, validation CER plateaued at a higher value for the same training duration. We

hypothesize that our model is already heavily regularized through dropout, data augmentation, and AdamW's weight decay, and that introducing additional regularization-based strategies consequently constrains the optimization process rather than enhancing generalization.

2.5.4 Data Augmentation Strategies

We increased the strength of the existing augmentations by raising the white noise standard deviation from 0.8 to 1.0 and the constant offset standard deviation from 0.2 to 0.25, following recent findings in neural speech decoding [1]. We also introduced temporal masking augmentation, inspired by [3], which randomly masks contiguous temporal chunks of neural activity during training. We experimented with two hyperparameters: the *time mask width*, which controls the maximum duration of each masked segment, and *timemaskprob*, which determines the probability of applying a temporal mask to a given training example.

After three trials sweeping these parameters, we found the ideal mask width and probability to be 20 and 0.5, respectively. These settings improved the model's robustness to temporal variability and missing data, leading to more stable validation performance.

2.5.5 Time Masking

We performed time-masking by masking several time bins (GRU) of each trial. Time masking is a data augmentation technique to process sequential data, thereby making it a relevant technique for neural sequence decoding. Time-masking selectively masks portions of time-axis data ensuring that the GRU model doesn't train with an over-reliance on specific-time segments of the sequential data. The goal is to promote more generalizable learning of features.

For all analyses, we set time masked width to 20 and time mask probability to 0.5 which is yielded with $M = 0.05$.

For the time-masked GRU, the masked bins get 0. We present an algorithm for Time-masking:

Algorithm 1 TIMEMASK(x, N, M)

Input: Trial $x \in \mathbb{R}^{L \times C}$ (L patches, C features);
N — number of masks
M — max-mask length as a fraction of trial length ($0 < M \leq 1$)
Output: Masked trial $\tilde{x}$
$F \leftarrow \lfloor M \cdot L \rfloor$; `// maximum mask length`
for $k \leftarrow 1$ **to** N **do**
  Sample $S \sim U(0, L - F)$; `// start index`
  Sample $D \sim U(0, F)$; `// mask length`
  $x[S : S + D] \leftarrow 0$
return $\tilde{x}$

2.6 Other Techniques Used

2.6.1 Batch Length

Early experiments revealed that the model had not fully converged after 10,000 batches. We extended training to 30,000 batches, which allowed the model to reach significantly better local minima and improve the final PER substantially, which will be further discussed in the results section.

2.6.2 Optimizations for Training Duration

Baseline experiments with no architectural changes except hyperparameter tuning revealed that training time per batch of approximately 0.77. We implemented Nvidia's Automatic Mixed Precision (AMP) [6] for Deep Learning framework with PyTorch allowing a significant speed up in math intensive operations such as linear layers by using Tensor Cores. In particular, a benefit of AMP that made us utilize the framework was the speed up in memory-limited operations by accessing half the bytes compared to single precision. This revealed an empirical observation of training time approximately between 0.49 to 0.52. However, a cost of this implementation was the significant

degradation in PER reported and a tendency for the model to be stuck (where CTC loss decreases but PER often oscillates or explodes).

As a result, we performed Gradient Clipping, a theoretically proven methodology to accelerate training performance to overcome the PER degradations from implementing Nvidia AMP. In particular, our goal was to minimize training time whilst maintaining performance. Doing so would be a huge improvement in total training time and a possible better PER as we finetune hyperparameters. In practice, this implementation proved to be far more challenging and often resulted in significant PER drops with subsequent explosions. Naturally, we adjusted learning rate schedules and other hyperparameters such as dropout but this implementation had a very small window of optimal performance which we were unable to reach. [8]

2.7 Unpromising Approaches

2.7.1 Diphone Shifting

One approach [5] augmented the training dataset with diphones instead of phonemes and achieved noteworthy improvements over baseline PER with a bidirectional GRU model. We hypothesized that decoding diphones would outperform decoding just phonemes, especially since neural activity in the brain often encodes the movement of articulators vs static phonemes. We expanded the output vocabulary from 40 phonemes to 913 diphone pairs and trained the baseline GRU on this target set. The model failed to converge to a competitive PER, performing significantly worse than the phoneme baseline.

One likely reason for this is that while common transitions are well represented, rare transitions appear only a handful of times in the dataset. In addition, CTC in particular does much better at identifying stable states versus transitions, making it harder for it to decode it. Diphone encoding is also less effective with unidirectional training as most of the value is lost. Lastly, the above paper uses a complicated divide and conquer strategy whereas this strategy simply treated the diphones as classes.

2.7.2 Feature Masking

We also hypothesized that feature masking would potentially allow the model to learn more distributed representations. However, unlike time masking, feature masking led to a slight degradement. Masking these channels removes critical information; unlike with an image where neighbors are correlated, one channel can contain far more information than the ones right next to it.

2.7.3 Log Transforms

In addition, we explored log-transforming the data before normalizing it, as prior work suggested it made some impact [3]. However, the performance was roughly the same, if not a bit worse. This may reflect the nature of unidirectional training versus bidirectional.

2.8 Final Model

Our final model integrates the most effective modifications identified during the experimental phase. The architecture follows the processing pipeline described below.

Input Processing As in the baseline model, 256-dimensional neural features are smoothed using a Gaussian convolution with width 2.0. Features are passed through affine transformations followed by a softsign nonlinearity to mitigate session-to-session variability.

Normalization We introduce a LayerNorm step after day-specific adaptation and before the GRU encoder to stabilize the feature distribution and reduce internal covariate shift, improving optimization stability.

GRU Encoder The recurrent encoder matches the project specification: a 5-layer unidirectional GRU with 1024 hidden units, kernel length 32, and stride 4. This stage mirrors the baseline configuration but benefits from the normalized input distribution.

Post-GRU Processing To address the limited representational capacity of the baseline decoder and improve downstream linear separability, we add a post-GRU processing block. This block consists of a linear layer that preserves the GRU output dimensionality (2048), followed by LayerNorm, a ReLU nonlinearity, and dropout with rate 0.35. These layers allow the model to refine and regularize high-level temporal representations prior to classification.

Output Layer The final linear projection to 40 phoneme classes (plus the CTC blank symbol) remains unchanged from the baseline system.

2.9 Final Training Procedure

The model is trained for $30{,}000$ batches with a batch size of 16 using the AdamW optimizer with weight decay 10^{-5}. We employ a learning-rate schedule consisting of 500 warmup steps from 0 to 0.02, followed by linear decay to 0.008 for the remainder of training.

Final data augmentation includes additive white noise with standard deviation 1.0, constant offset shifts with standard deviation 0.25, and temporal masking with a maximum width of 20 time steps applied with probability 0.5. Dropout with rate 0.35 is applied to both the GRU and post-GRU layers to improve generalization.

3 Results

3.1 Comparison with Baseline Unidirectional GRU

The baseline GRU model, trained for 10,000 batches with the standard configuration described in the Methods section, achieves a validation Phoneme Error Rate (PER) of 0.226817. This serves as our reference point for evaluating improvements. Our final model, trained for 30,000 batches, achieves a validation PER of 0.189757 which corresponds to a 16.3% relative improvement over the baseline PER. The baseline model runs at an average of 0.893 seconds per batch, whereas our final model

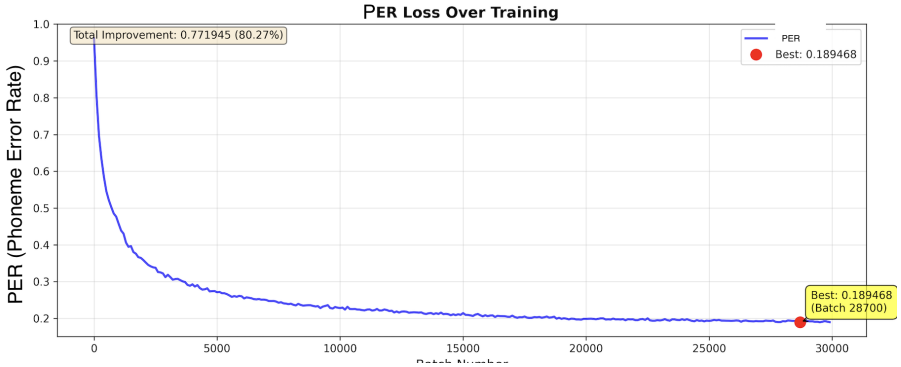

Figure 1: Validation PER over 30,000 training batches for the final model. The model achieves its best PER of 0.1895 at batch 28,700.

runs at a slightly higher average of 0.956 seconds per batch, showing that not much time (per batch) has to be sacrificed in order to get such improved results. However, we must note that increasing the number of batches by threefold will make the total training significantly longer.

3.2 Comparison against Top-Performing Benchmark Entries

We now compare the final model with the top-performing entries in the Brain-to-Text Benchmark '24. This allows a contextualization of the architectural enhancements made to the baseline GRU model and the strength of our methodologies in comparison with other leading implementations over the dataset. The top performing benchmark entry presented by Shlizerman Lab of the University of Washington[5] achieves a Phoneme Error Rate (PER) of 15.34% with the use of diphones and Large Language Model based post-processing in comparison to monophones to capture context-aware representations of neural data while still producing phoneme-level outputs. In particular, the

Shlizerman Lab keeps the same GRU architecture as we present in this paper, however, they utilize an ensemble strategy where they feed both phoneme outputs and candidate word-sequence hypotheses into a Large Language Model allowing it to produce the most plausible transcription. This strategy is further tuned with DCoNDF-LI and DCoND-LIFT where the Lab uses in-context learning and LLM fine-tuning based on decoding output respectively.

Without adding model complexity, through architectural changes and finetuning, we are able to achieve a PER of 18.95%. In particular, our architectural changes can significantly benefit from a diphone decoding and divide-and-conquer strategy as presented by the Shlizerman Lab allowing a reduction in phoneme misclassification.

4 Discussion

Initially, to see what would and wouldn't work, a wide array of optimizations were tried. However, as multiple changes were combined at the same time, model improvement over the baseline started to stagnate. For a unidirectional model, the learning signal is much more sparse, and many overlapping filters can limit the model from learning the data.

The bulk of the improvements came mainly from adjusting an architecture optimized for bidirectional training to one better suited for unidirectional training. For example, bidirectional models tend to converge quickly due to gaining additional context from information that comes after. Unidirectional models tend to be less stable and improve noticeably with more rounds of training. While 10000 batches was enough to reach convergence for the bidirectional GRU, expanding to 30000 batches led to the model fitting the data better.

In addition, the minimal focus on day-specific transformations in the original architecture was suitable for a bidirectional model where noise could be smoothened out by context. However, for a unidirectional model, much of the noise from day-to-day differences persisted. Implementing Layer Normalization helped reduce session specific noise and increase performance.

For additional improvements, some potential options include ensembling models trained at different seeds, distilling a model trained on bidirectional to improve unidirectional performance, and incorporating a robust transformer architecture. In addition, recent advancements in LLMs could lead to significant WER improvements, especially if focusing on the more general-purpose set of phrases and words many patients need versus the broader language.

5 Acknowledgment

AD, ND and AU acknowledge support from Google on providing Google Cloud Platform compute credits enabling this paper's results for model training and evaluation. AD, NH and AU also acknowledge support from Dr. Jonathan Kao (kao@seas.ucla.edu), Shreeram Athreya shreeram@g.ucla.edu and Mehmet Yigit Turali myigitturali@g.ucla.edu on Neural Signal Processing & Brain-Computer Interface with Machine Learning.

References

[1] Nicholas S Card, Maitreyee Wairagkar, Carrina Iacobacci, Xianda Hou, Tyler Singer-Clark, Francis R Willett, Erin M Kunz, Chaofei Fan, Maryam Vahdati Nia, Darrel R Deo, Aparna Srinivasan, Eun Young Choi, Matthew F Glasser, Leigh R Hochberg, Jaimie M Henderson, Kiarash Shahlaie, Sergey D Stavisky, and David M Brandman. An accurate and rapidly calibrating speech neuroprosthesis. *New England Journal of Medicine*, 391(7):609–618, August 2024.

[2] Umberto Dall'Asen, Sergey D. Stavisky, Francis R. Willett, Sinem Fok, Mostafa Azimipour, Marissa Darby, Michelle Leavitt, Jonathan C. Kao, Daniel J. O'Shea, Karunesh Ganguly, and Krishna V. Shenoy. The brain2text benchmark: A standardized evaluation suite for neural speech decoding from intracortical signals. *bioRxiv*, 2024.

[3] Ebrahim Feghhi, Shreyas Kaasyap, Nima Hadidi, and Jonathan C Kao. Time-masked transformers with lightweight test-time adaptation for neural speech decoding. *arXiv preprint arXiv:2507.xxxxx*, July 2025.

[4] Yanzhang He, Tara N. Sainath, Rohit Prabhavalkar, Ian McGraw, Raziel Alvarez, Ding Zhao, David Rybach, Anjuli Kannan, Yonghui Wu, Ruoming Pang, Qiao Liang, Deepti Bhatia, Yuan Shangguan, Bo Li, Golan Pundak, Khe Chai Sim, Tom Bagby, Shuo-yiin Chang, Kanishka Rao, and Alexander Gruenstein. Streaming end-to-end speech recognition for mobile devices. (arXiv:1811.06621), November 2018. arXiv:1811.06621 [cs].

[5] Jingyuan Li, Trung Le, Chaofei Fan, Mingfei Chen, and Eli Shlizerman. Brain-to-text decoding with context-aware neural representations and large language models. *arXiv preprint*, arXiv:2411.10657, 2024.

[6] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. (arXiv:1710.03740), February 2018. arXiv:1710.03740 [cs].

[7] Francis R. Willett, Erin M. Kunz, Chaofei Fan, Donald T. Avansino, Guy H. Wilson, Eun Young Choi, Foram Kamdar, Matthew F. Glasser, Leigh R. Hochberg, Shaul Druckmann, Krishna V. Shenoy, and Jaimie M. Henderson. A high-performance speech neuroprosthesis. *Nature*, 620(7976):1031–1036, August 2023.

[8] Jingzhao Zhang, Tianxing He, Suvrit Sra, and Ali Jadbabaie. Why gradient clipping accelerates training: A theoretical justification for adaptivity. (arXiv:1905.11881), February 2020. arXiv:1905.11881 [math].

A Computational Resources

The majority of our results were generated using one Nvidia T4 GPU and a Google Compute Engine Machine Type n1-standard-4 with 15GB Memory. The CPU platform was Intel Skylake.

B Hyperparameters

We list the hyperparameters for the baseline GRU in Table 1 and for the enhanced GRU in Table 2.

Table 1: Baseline GRU Hyperparameters

Hyperparameter	**Value**
Stride Size	4 (80 ms)
Layers	5
Hidden Size	1024
Input Features	256
Sequence Length	150
Batch Size	64
Total Batches	10000
LR	0.02
White Noise	0.8
Baseline Shift	0.2
L2 Decay	$1e{-5}$
Dropout	0.4
Gaussian Smooth Kernel Size	32
Gaussian Smooth σ	2.0
Adam Optimizer ε	0.1
Bidirectional	False

Table 2: Enhanced GRU Hyperparameters

Hyperparameter	**Value**
Stride Size	4 (80 ms)
Layers	5
Hidden Size	1024
Input Features	256
Sequence Length	150
Batch Size	16
Total Batches	30000
LR	0.02
White Noise	1.0
Baseline Shift	0.25
L2 Decay	$1e{-5}$
Dropout	0.35
Gaussian Smooth Kernel Size	32
Gaussian Smooth σ	2.0
Adam Optimizer ε	$1e{-5}$
Bidirectional	True